SRM UNIVERSITY AP

School of Engineering and Applied Sciences

Department of Computer Science and Engineering

MINOR PROJECT REPORT

AI HEALTHCARE SYMPTOM ASSESSMENT SYSTEM

Machine Learning Based Symptom Assessment

Submitted by

AKHIL MANOHAR ANNAMAREDDY

B.Tech – Computer Science and Engineering

Academic Year: 2024–2028

CERTIFICATE

This is to certify that **AKHIL MANOHAR ANNAMAREDDY**, a student of **B.Tech – Computer Science and Engineering**, Department of Computer Science and Engineering, **SRM UNIVERSITY AP**, has successfully completed the Minor Project titled **"AI HEALTHCARE SYMPTOM ASSESSMENT SYSTEM"** as part of the academic requirements for the academic year **2024–2028**.

The project work demonstrates the application of machine learning techniques for symptom-based assessment and has been completed in accordance with the prescribed academic requirements of the Department of Computer Science and Engineering.

Department of Computer Science and Engineering
SRM UNIVERSITY AP

DECLARATION

I, **AKHIL MANOHAR ANNAMAREDDY**, a student of **B.Tech – Computer Science and Engineering**, Department of Computer Science and Engineering, **SRM UNIVERSITY AP**, hereby declare that the Minor Project entitled **"AI HEALTHCARE SYMPTOM ASSESSMENT SYSTEM"** is my original work carried out as part of the academic requirements for the academic year **2024–2028**.

I further declare that this project has been developed using machine learning techniques for symptom-based assessment and that the work presented in this report has been completed by me to the best of my knowledge and understanding.

I also declare that this project report has not been submitted, either in whole or in part, to any other institution or university for the award of any degree, diploma, or other academic qualification.

AKHIL MANOHAR ANNAMAREDDY

B.Tech – Computer Science and Engineering

SRM UNIVERSITY AP

ACKNOWLEDGEMENT

I would like to express my sincere thanks to everyone who supported me in completing my minor project, “AI HEALTHCARE SYMPTOM ASSESSMENT SYSTEM.”

I am grateful to the Department of Computer Science and Engineering, SRM UNIVERSITY AP, for giving me the opportunity to work on this project and gain practical experience in the field of machine learning.

This project helped me understand how machine learning can be used with real-world datasets to build a simple symptom assessment system. While working on the project, I learned about data preprocessing, feature engineering, model training, model evaluation, and the development of a basic web application using Streamlit.

I would also like to thank my faculty members and classmates for their suggestions, discussions, and support during the development of this project.

Finally, I would like to thank my family and friends for their constant encouragement and support throughout the completion of this project.

AKHIL MANOHAR ANNAMAREDDY

B.Tech – Computer Science and Engineering

SRM UNIVERSITY AP

ABSTRACT

The AI Healthcare Symptom Assessment System is a machine learning based project developed to provide a simple way for users to assess possible health conditions based on the symptoms they enter. The main idea behind the project is to use a machine learning model to identify a possible condition from a given set of symptoms.

For this project, a disease and symptom dataset containing different diseases and their associated symptoms was used. The data was cleaned and prepared before training the machine learning models. The symptoms were converted into numerical features using multi-hot encoding, where each symptom is represented as either present or absent. Different machine learning algorithms, including Decision Tree, Random Forest, Extra Trees, and K-Nearest Neighbors, were trained and compared.

Based on the model evaluation, Random Forest was selected as the final model for the application. The final model was trained using the processed symptom data and saved for use in the application. A simple web interface was then developed using Streamlit, allowing users to enter their symptoms and receive a possible condition along with a prediction score and basic educational information.

The project demonstrates how machine learning can be applied to a healthcare-related dataset to build a basic symptom assessment system. However, the system is intended only for educational and project purposes. It is not a medical diagnostic tool, and its predictions should not be considered a substitute for professional medical advice.

Keywords: Machine Learning, Healthcare, Symptom Assessment, Random Forest, Disease Prediction, Multi-Hot Encoding, Streamlit

TABLE OF CONTENTS

Contents

INTRODUCTION 7
PROBLEM STATEMENT 8
OBJECTIVES 9
EXISTING SYSTEM 10
PROPOSED SYSTEM 11
SYSTEM ARCHITECTURE 12
DATASET DESCRIPTION 14
DATA PREPROCESSING 15
FEATURE ENGINEERING 17
MACHINE LEARNING ALGORITHMS 18
MODEL TRAINING 20
MODEL EVALUATION 22
RESULTS AND DISCUSSION 25
STREAMLIT APPLICATION 28
ADVANTAGES 30
LIMITATIONS 31
FUTURE SCOPE 33
CONCLUSION 34
REFERENCES 35

INTRODUCTION

Healthcare is an important part of everyday life, and people often experience different symptoms such as fever, headache, itching, cough, stomach pain, or skin problems. When several symptoms occur together, it can sometimes be difficult for a person to understand what they may be related to. With the development of machine learning, data can be used to identify patterns and provide useful information based on previously available examples.

The AI Healthcare Symptom Assessment System is a minor project developed to explore how machine learning can be used for symptom-based assessment. The system takes a set of symptoms entered by the user and compares them with patterns learned from a dataset. Based on these patterns, the trained model provides a possible condition along with a prediction score.

The project uses a dataset containing different diseases and their associated symptoms. Before training the models, the dataset is cleaned and prepared so that it can be used effectively by machine learning algorithms. Since machine learning models work with numerical data, the symptoms are converted into binary features using multi-hot encoding. This allows the model to understand whether a particular symptom is present or absent.

Several machine learning algorithms are considered in the project, including Decision Tree, Random Forest, Extra Trees, and K-Nearest Neighbors. Their performance is compared using standard evaluation measures, and Random Forest is selected as the final model for the application.

A simple web application is developed using Streamlit to make the system easier to use. Users can enter their symptoms through the application and view the model's prediction along with basic educational information. The application is designed as a learning and demonstration project rather than a replacement for a doctor or other healthcare professional.

Overall, this project provides practical experience in data preprocessing, machine learning, model evaluation, and application development. It also shows how machine learning concepts learned in a computer science course can be applied to a real-world problem area such as healthcare.

PROBLEM STATEMENT

People can experience many different symptoms during an illness, and some symptoms may be common to more than one health condition. Understanding the possible relationship between a group of symptoms and a particular condition can therefore be difficult, especially when a person does not have enough knowledge about the symptoms.

Traditional symptom checking usually depends on manually searching for symptoms and comparing them with information from different sources. This process can take time and may also lead to confusion when several conditions have similar symptoms.

The problem addressed in this project is to develop a simple machine learning based system that can take a group of symptoms as input and identify a possible condition based on patterns present in a prepared dataset. The system should be easy to use and should provide the result in a simple and understandable format.

The project also focuses on comparing different machine learning algorithms to determine which model performs better on the given dataset. The selected model is then integrated into a Streamlit application so that users can enter symptoms without directly interacting with the machine learning code.

The system is not intended to provide a medical diagnosis. Its purpose is to demonstrate how machine learning can be applied to a healthcare-related dataset for educational and academic purposes. Any actual health concern should be evaluated by a qualified healthcare professional.

OBJECTIVES

The main objective of this project is to develop a simple machine learning based system that can assess a user's symptoms and provide a possible health condition based on the patterns learned from the dataset.

The specific objectives of the project are:

• To understand and prepare a healthcare-related dataset containing diseases and their associated symptoms.

• To clean the dataset by handling empty values and removing unnecessary duplicate records.

• To convert the symptoms into a suitable numerical format using multi-hot encoding.

• To train different machine learning models and compare their performance on the given dataset.

• To select a suitable machine learning model for the final symptom assessment system.

• To save the trained model and use it for making predictions on new symptom inputs.

• To develop a simple and user-friendly web application using Streamlit.

• To display the predicted possible condition, prediction score, and basic educational information in an understandable way.

• To understand the practical steps involved in developing a machine learning project, from data preparation to application development.

• To clearly communicate that the system is intended for educational purposes and should not be treated as a replacement for professional medical advice.

EXISTING SYSTEM

At present, people generally use different methods to understand the possible reason behind their symptoms. One common method is to search for individual symptoms on the internet and compare the information with different health conditions. Although this can provide useful information, it may also be difficult to understand when several symptoms are present at the same time.

Another common approach is to consult a doctor or healthcare professional. This is the most appropriate option for an actual medical concern because a healthcare professional can consider the person's medical history, physical condition, test results, and other important factors. However, professional consultation may not always be immediately available when a person simply wants basic information about a group of symptoms.

Many online symptom-checking systems are also available. These systems may use predefined rules, databases, or more advanced algorithms to provide possible conditions based on user inputs. However, such systems can vary in their approach, complexity, and the amount of information they provide.

The existing approaches have some limitations when considered from the perspective of a simple academic machine learning project. Manual searching can take time, rule-based systems may have limited flexibility, and complex systems may not be easy for students to understand or modify.

Therefore, this project focuses on developing a simple machine learning based symptom assessment system. The aim is not to replace existing healthcare services, but to demonstrate how a machine learning model can learn patterns from a symptom dataset and provide a possible condition based on user input.

PROPOSED SYSTEM

The proposed system is a machine learning based application that takes a user's symptoms as input and provides a possible health condition based on the patterns learned from the dataset. The main purpose of the system is to demonstrate the use of machine learning in a simple healthcare-related application.

The system starts with a dataset containing different diseases and their associated symptoms. The dataset is first checked and cleaned to remove unnecessary or repeated records. The symptoms are then converted into numerical features so that they can be understood by machine learning algorithms.

Different machine learning models are trained using the processed data. In this project, Decision Tree, Random Forest, Extra Trees, and K-Nearest Neighbors algorithms are considered and their results are compared. Based on the evaluation, Random Forest is used as the final model for the application.

The trained model is saved and connected to a Streamlit application. In the application, the user can enter or select symptoms. The entered symptoms are converted into the same feature format used during training and passed to the trained model. The model then provides a possible condition and a prediction score.

The application also provides basic educational information related to the predicted condition. A clear message is included to explain that the result is only an assessment based on the dataset and should not be considered a medical diagnosis.

The proposed system provides a simple way to demonstrate the complete machine learning workflow, starting from data preparation and model training and ending with a working application that can be used through a web browser.

SYSTEM ARCHITECTURE

The system is designed as a simple sequence of steps, starting with the user's symptoms and ending with a possible condition generated by the trained machine learning model. Each step has a specific role in preparing the input, making the prediction, and displaying the result.

The main components of the system are described below.

1. USER INPUT

The user enters or selects the symptoms that they are experiencing through the Streamlit application. The system accepts multiple symptoms so that the model can consider the combination of symptoms rather than only one symptom.

2. SYMPTOM PROCESSING

The symptoms entered by the user are checked against the list of symptoms used during model training. The selected symptoms are converted into the required feature format.

3. FEATURE REPRESENTATION

The system uses multi-hot encoding to represent symptoms. Each symptom is treated as a separate feature. A value of 1 indicates that the symptom is present, while a value of 0 indicates that it is not present.

4. MACHINE LEARNING MODEL

The processed symptom features are given to the trained Random Forest model. The model uses patterns learned during training to identify the condition that best matches the given symptom combination.

5. PREDICTION RESULT

The model produces a possible condition along with a prediction score. The application also displays the top three predictions so that the user can see the other conditions considered by the model.

6. INFORMATION DISPLAY

After the prediction, the application displays basic educational information related to the predicted condition. General advice and a clear medical disclaimer are also shown.

7. SYSTEM OUTPUT

The final output is presented through the Streamlit interface in a simple format. The result is intended only for educational and demonstration purposes and should not be treated as a medical diagnosis.

SYSTEM FLOW

User Symptoms
↓
Input Processing
↓
Multi-Hot Feature Encoding
↓
Trained Random Forest Model
↓
Prediction and Prediction Score
↓
Top 3 Predictions
↓
Educational Information and General Advice
↓
Medical Disclaimer

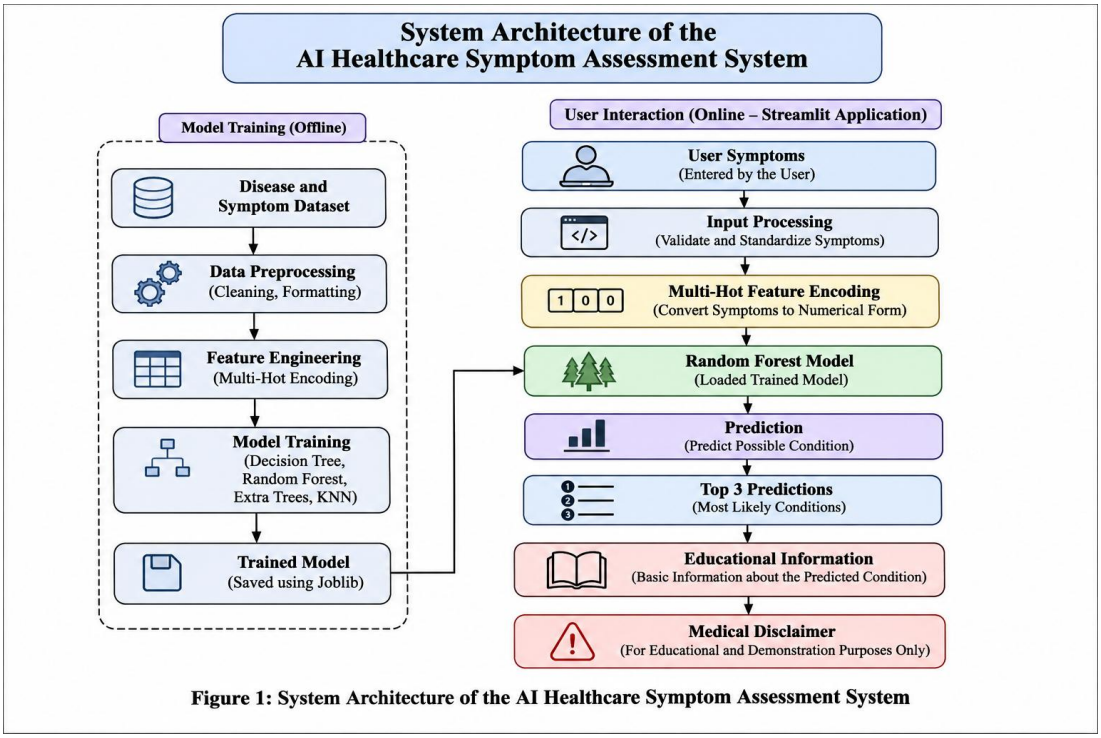


Figure 1: System Architecture of the AI Healthcare Symptom Assessment System

DATASET DESCRIPTION

The dataset used in this project contains information about different diseases and the symptoms associated with them. It was used to train and test the machine learning models developed for the symptom assessment system.

The original dataset contains **4,920 records** and **18 columns**. The first column represents the disease, while the remaining columns contain the symptoms associated with each record. The symptom columns are named from **Symptom_1** to **Symptom_17**.

The dataset contains **41 different disease classes** and approximately **131 unique symptoms**. Each record represents a combination of symptoms related to a particular disease. Since the same symptoms can occur in different conditions, the machine learning model learns patterns from combinations of symptoms rather than relying on only one symptom.

Dataset Structure

Dataset Property	**Details**
Total original records	4,920
Total columns	18
Target column	Disease
Symptom columns	Symptom_1 to Symptom_17
Number of disease classes	41
Unique symptoms	Approximately 131
Feature representation	Multi-hot encoding

The dataset required some cleaning before it could be used for model training. Empty records and exact duplicate records were removed during

preprocessing. After cleaning, the remaining records were used for creating the machine learning features and training the models.

The disease column was treated as the target variable because it represents the condition that the model needs to predict. The symptom columns were used as input information for the model.

Since machine learning algorithms require numerical input, the symptoms were converted into binary features. This process is explained in the next section.

The dataset is suitable for demonstrating the basic working of a symptom-based machine learning system. However, it should not be considered a clinically validated healthcare dataset, and the results of the project should not be used for actual medical diagnosis.

DATA PREPROCESSING

Data preprocessing is an important step in this project because the original dataset cannot be directly given to a machine learning model. The symptom data contains empty values, repeated records, and text-based symptom names. The data was therefore cleaned and converted into a suitable format before training the models.

The following steps were carried out during preprocessing.

8.1 Loading the Dataset

The dataset was loaded using the Pandas library in Python. After loading the file, the number of records, columns, and the structure of the dataset were checked to understand the available information.

8.2 Checking Missing Values

The symptom columns were checked for empty or missing values. Since some records did not contain all possible symptoms, the empty values were handled during the cleaning process so that they would not create problems during feature creation.

8.3 Removing Unnecessary Records

Records that did not contain useful information were removed. Exact duplicate records were also removed to reduce unnecessary repetition in the data.

After cleaning, the dataset contained fewer records than the original dataset. This cleaned dataset was then used for creating the final feature matrix.

8.4 Cleaning Symptom Names

The symptom names were collected from all symptom columns and processed to create a common list of symptoms. This ensured that the same symptom was treated as one feature even if it appeared in different records.

8.5 Converting Symptoms into Numerical Features

Machine learning models cannot directly work with symptom names written as text. Therefore, each unique symptom was converted into a separate feature.

A value of **1** was assigned when a symptom was present in a particular record, while a value of **0** was assigned when the symptom was absent. This method is called multi-hot encoding.

8.6 Preparing the Target Variable

The disease column was used as the target variable. Since disease names are text values, they were converted into numerical labels using Label Encoding. The encoded labels were used during model training, while the original disease names were retained for displaying the prediction results.

8.7 Final Prepared Data

After preprocessing, the dataset was in a form that could be directly used by the machine learning algorithms. The input consisted of binary symptom features, while the output was the encoded disease label.

The preprocessing step helped create a consistent input format for all the models used in the project and also ensured that the same feature representation could be used when making predictions through the Streamlit application.

FEATURE ENGINEERING

Feature engineering is the process of converting the available data into a format that can be used effectively by a machine learning model. In this project, the main input information is the set of symptoms associated with each disease.

Since the original symptoms are stored as text, they cannot be directly used by most machine learning algorithms. Therefore, the symptoms were converted into numerical features using a multi-hot encoding approach.

9.1 Multi-Hot Encoding

In multi-hot encoding, every unique symptom is treated as a separate feature. For each record, the value of a feature is either 1 or 0.

A value of **1** means that the particular symptom is present in the record, while a value of **0** means that the symptom is not present.

For example, if the available symptoms are:

Fever, Cough, Headache, Itching

and a particular record contains:

Fever, Cough, Headache

the corresponding feature representation would be:

Fever	Cough	Headache	Itching
1	1	1	0

This representation allows the machine learning model to work with the symptoms as numerical input.

9.2 Symptom Feature List

All unique symptoms found in the cleaned dataset were collected into a single feature list. This list was saved as **symptom_features.pkl** so that the same

feature order could be used when the Streamlit application receives new symptoms.

Maintaining the same feature order is important because the trained model expects the input features in the same format that was used during training.

9.3 Target Encoding

The disease column is the output that the model needs to predict. Since the disease names are text values, **LabelEncoder** was used to convert them into numerical labels.

For example, different disease names can be internally represented by numbers such as 0, 1, 2, and so on. The trained label encoder was saved as **label_encoder.pkl** so that the numerical prediction could later be converted back to the original disease name.

9.4 Importance of Feature Engineering

The feature engineering step connects the original dataset with the machine learning algorithms. By converting the symptoms into a consistent numerical representation, the same type of input can be used during training, testing, and prediction.

This approach also makes it easier to save the trained model and use it later in the Streamlit application without repeating the complete training process.

MACHINE LEARNING ALGORITHMS

In this project, different machine learning algorithms were trained and compared to find a suitable model for symptom-based assessment. The algorithms were selected because they can work with numerical feature data and are commonly used for classification problems.

The four algorithms used in this project are Decision Tree, Random Forest, Extra Trees, and K-Nearest Neighbors.

10.1 Decision Tree

A Decision Tree is a classification algorithm that makes decisions by dividing the data into smaller groups based on feature values. It works in the form of a tree, where the internal nodes represent decisions, the branches represent possible outcomes, and the final nodes represent the predicted class.

In this project, the symptoms are used as input features and the disease is the target class. The model learns which symptom combinations are useful for separating different disease classes.

Decision Trees are easy to understand and visualize, but a single tree can sometimes become too closely fitted to the training data.

10.2 Random Forest

Random Forest is an ensemble learning algorithm that uses multiple decision trees instead of relying on only one tree. Each tree makes a prediction, and the final prediction is obtained by combining the results of the individual trees.

Random Forest was selected as the final model in this project. The final model was configured with **300 decision trees** and a random state of **42**.

One advantage of Random Forest is that using multiple trees can make the model more stable than using a single Decision Tree. It is also suitable for classification problems with multiple classes.

10.3 Extra Trees

Extra Trees, also known as Extremely Randomized Trees, is another ensemble-based classification algorithm. It creates multiple decision trees and introduces additional randomness while selecting the splits used by the trees.

The use of several randomized trees can help the model capture different patterns in the training data. Extra Trees was included in the project to compare its performance with Random Forest and Decision Tree.

10.4 K-Nearest Neighbors

K-Nearest Neighbors, or KNN, makes predictions by looking at the data points that are closest to a new input. The class of the new input is determined based on the classes of its nearest neighboring data points.

For this project, the symptom features are used to determine the similarity between records. The disease associated with similar records can then be used to make a prediction.

KNN is simple to understand, although its performance can depend on the amount and structure of the available data.

10.5 Algorithm Comparison

All four algorithms were trained using the same processed symptom features and target labels. Their results were then compared using classification metrics such as accuracy, precision, recall, and F1-score.

The comparison helped in selecting Random Forest as the final model used in the Streamlit application.

Algorithm	Type	Main Idea
Decision Tree	Classification	Makes decisions using a tree structure
Random Forest	Ensemble Classification	Combines multiple decision trees
Extra Trees	Ensemble Classification	Uses multiple randomized decision trees
KNN	Classification	Predicts using nearby data points

MODEL TRAINING

After completing the data preprocessing and feature engineering steps, the prepared dataset was used to train the machine learning models. The main purpose of training was to allow each model to learn the relationship between the symptoms and the corresponding disease classes.

The dataset was divided into training and testing portions. The training data was used to build the models, while the testing data was kept separate to evaluate how the models performed on data that was not used during training.

11.1 Train-Test Split

An **80:20 train-test split** was used in the project. Approximately 80% of the prepared records were used for training and the remaining 20% were used for testing.

A fixed random state of **42** was used so that the same split could be reproduced when the notebook was executed again. Stratification was also used to maintain the distribution of the different disease classes between the training and testing sets.

11.2 Training the Models

Four classification models were trained using the same training data:

- Decision Tree
- Random Forest
- Extra Trees
- K-Nearest Neighbors

Each model received the multi-hot encoded symptom features as input and the encoded disease labels as the target.

The models were then evaluated using the testing data. Their predictions were compared with the actual disease labels to calculate the evaluation metrics.

11.3 Random Forest Configuration

Random Forest was selected as the final model after comparing the performance of the different algorithms.

The final Random Forest model was configured with:

Parameter	Value
Algorithm	Random Forest Classifier
Number of trees	300

Random state	42
Input	Multi-hot encoded symptoms
Output	Disease class

The final Random Forest model was then trained using the complete cleaned dataset so that it could make predictions when used by the Streamlit application.

11.4 Saving the Trained Model

After training, the final model was saved using the Joblib library. This avoids the need to train the model every time the application is started.

Three important files were saved:

disease_model.pkl – contains the trained Random Forest model.

label_encoder.pkl – converts the model's numerical output back into the original disease name.

symptom_features.pkl – contains the list of symptom features used by the model.

These files are loaded by the Streamlit application when a user makes a prediction.

MODEL EVALUATION

After training the machine learning models, their performance was evaluated using the testing data. Model evaluation helps to understand how well each algorithm performs when predicting the different disease classes in the dataset.

In this project, four machine learning models were compared: Decision Tree, Random Forest, Extra Trees, and K-Nearest Neighbors.

12.1 Evaluation Metrics

The following metrics were used to compare the performance of the models.

Accuracy

Accuracy represents the percentage of predictions that were correctly classified out of all the predictions made by the model.

Accuracy = Correct Predictions / Total Predictions

Precision

Precision indicates how many of the samples predicted as a particular class actually belonged to that class. It gives an idea of how reliable the model's positive predictions are.

Recall

Recall indicates how many of the actual samples belonging to a particular class were correctly identified by the model.

F1-Score

F1-score combines precision and recall into a single measure. It provides a balanced view of the model's performance when both precision and recall are important.

Since this project contains multiple disease classes, weighted averages were used for precision, recall, and F1-score.

12.2 Model Comparison

The four models were trained and evaluated using the prepared dataset. The results obtained from the evaluation are shown below.

Table 5: Performance Comparison of Machine Learning Models

Model	Accuracy	Precision	Recall	F1-Score
Decision Tree	65.57%	67.12%	65.57%	63.72%
Random Forest	100.00%	100.00%	100.00%	100.00%

Extra Trees	100.00%	100.00%	100.00%	100.00%
K-Nearest Neighbors	100.00%	100.00%	100.00%	100.00%

The results show that Random Forest, Extra Trees, and K-Nearest Neighbors achieved 100% accuracy, precision, recall, and F1-score on the evaluated dataset. Decision Tree achieved lower scores compared with the other three models.

Random Forest was selected as the final model for the application. It provides an ensemble-based approach using multiple decision trees and performed very well on the given dataset.

12.3 Confusion Matrix

A confusion matrix was generated to examine the predictions made by the models across the different disease classes. It provides a visual representation of correctly and incorrectly classified samples.

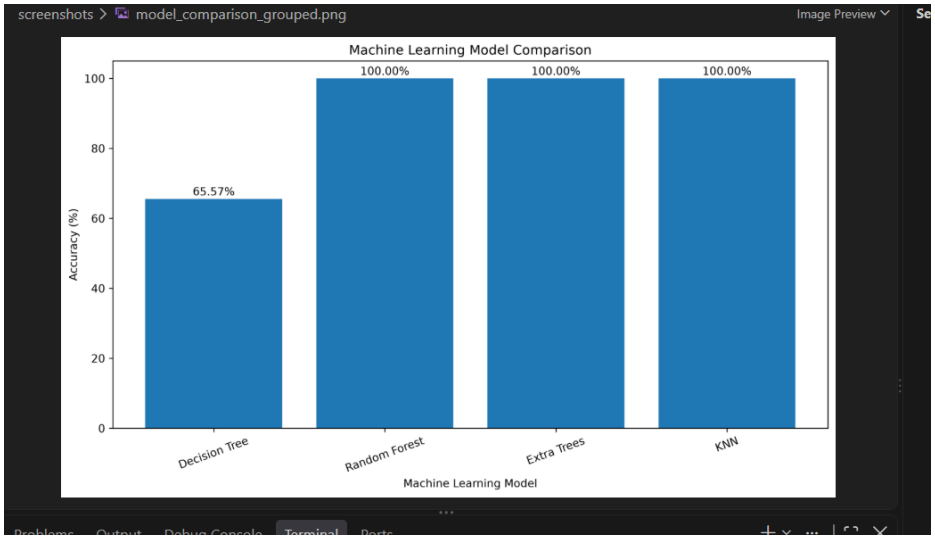
The confusion matrix generated during the project is included in the Results section of this report.

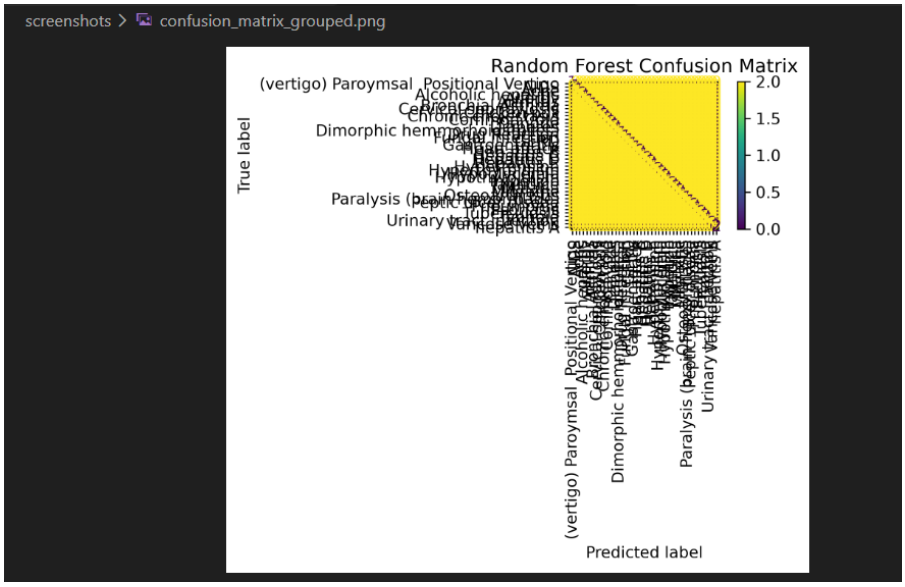
12.4 Interpretation of the Results

The high scores obtained by Random Forest, Extra Trees, and K-Nearest Neighbors should be interpreted carefully. The dataset contains repeated and very similar symptom patterns, and the cleaned dataset is relatively small compared with the original dataset.

Therefore, the 100% scores represent the performance of these models on the particular dataset and evaluation setup used in this project. They should not be interpreted as 100% accuracy for real-world healthcare or medical diagnosis.

The models have not been clinically validated. The system is intended for educational and academic purposes and should not be used to make actual medical decisions.





RESULTS AND DISCUSSION

The machine learning models developed in this project were evaluated using the prepared symptom dataset. The main purpose of the evaluation was to compare the performance of different classification algorithms and select a suitable model for the final application.

The results showed a clear difference between the Decision Tree and the other three models. The Decision Tree achieved an accuracy of 65.57%, with a precision of 67.12%, recall of 65.57%, and F1-score of 63.72%.

Random Forest, Extra Trees, and K-Nearest Neighbors achieved 100% accuracy, precision, recall, and F1-score on the evaluated dataset. Based on these results, all three models performed very well on the available test data.

13.1 Model Performance

The Decision Tree produced comparatively lower results. This indicates that a single decision tree was not able to classify the prepared symptom patterns as effectively as the other models.

Random Forest performed very well because it combines the predictions of multiple decision trees. Extra Trees also produced strong results because it uses multiple randomized decision trees. K-Nearest Neighbors achieved the same scores on the evaluated dataset by identifying similar symptom patterns in the feature space.

Although several models achieved 100% scores, these results need to be understood in the context of the dataset. The dataset contains repeated and similar symptom combinations, which can make classification easier. Therefore, the results should not be considered as evidence that the system would achieve 100% accuracy with real patients or real-world medical data.

13.2 Final Model Selection

Random Forest was selected as the final model for the application. It was chosen because it performed strongly during evaluation and provides a reliable ensemble-based classification approach.

The final Random Forest model was trained using the cleaned dataset and configured with 300 trees. The trained model was then saved and connected to the Streamlit application.

13.3 Prediction Output

The final application was tested by entering different combinations of symptoms. The system successfully processed the symptoms, converted them into the required feature format, and passed them to the trained model.

The application displays a possible condition, a model prediction score, and the top three predictions. It also provides basic educational information and general advice related to the result.

The prediction score shown by the application represents the model's output for the given input. It should not be interpreted as the probability that a person actually has a particular medical condition.

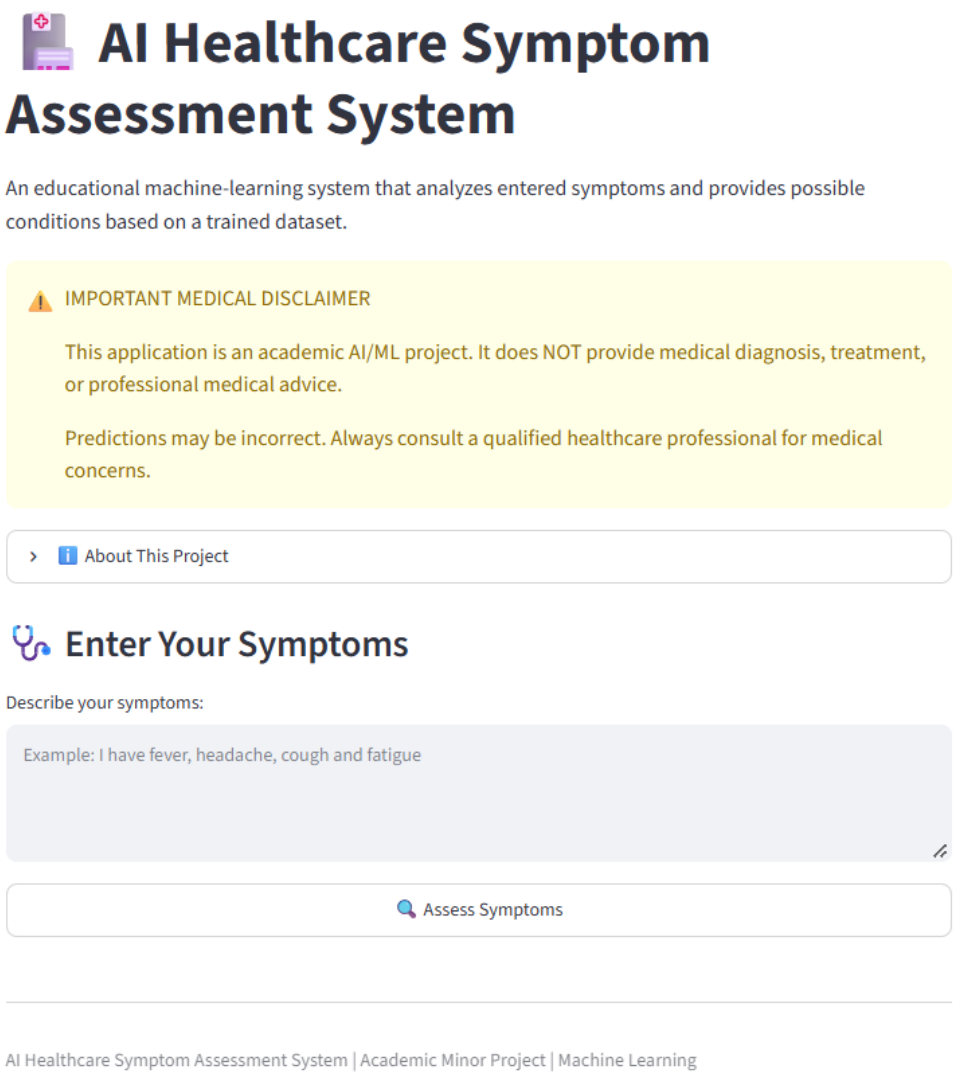
13.4 Overall Discussion

The project successfully demonstrates the complete process of developing a machine learning based symptom assessment application. It includes data cleaning, feature

engineering, model training, model comparison, model saving, and application development.

The results also show the importance of understanding the dataset before interpreting model performance. High evaluation scores can be influenced by the structure and repetition within a dataset. For this reason, further testing with larger, diverse, and clinically validated data would be necessary before considering such a system for any real healthcare use.

Main application screen



STREAMLIT APPLICATION

To make the machine learning model easier to use, a simple web application was developed using Streamlit. Streamlit is a Python-based framework that allows machine learning models to be connected to an interactive web interface without requiring a separate frontend development framework.

The application loads the trained Random Forest model along with the saved label encoder and symptom feature list. When the application starts, these files are loaded so that the model can be used directly without training it again.

14.1 User Interface

The application provides a simple interface where the user can enter or select symptoms. Multiple symptoms can be selected at the same time so that the model can consider the combination of symptoms.

The selected symptoms are converted into the same multi-hot format that was used during model training. This ensures that the input provided to the model has the correct number and order of features.

14.2 Prediction

After the symptoms are entered, the application sends the processed input to the trained Random Forest model. The model then returns a possible condition based on the patterns learned from the dataset.

The application displays the following information:

- Possible condition
- Model prediction score
- Top three predictions
- Basic educational information about the predicted condition
- General advice
- Medical disclaimer

14.3 Prediction Score

The application displays a prediction score along with the predicted condition. This score represents the model's output for the given input based on the available dataset.

The score should not be interpreted as a medical probability or as confirmation that a person has a particular disease. It is only a machine learning model output.

14.4 Safety and Disclaimer

Since this project is related to healthcare, a clear disclaimer is included in the application. The system is intended only for educational and demonstration purposes.

The application does not replace a doctor, medical examination, laboratory testing, or professional healthcare advice. Users with actual health concerns should consult a qualified healthcare professional.

14.5 Running the Application

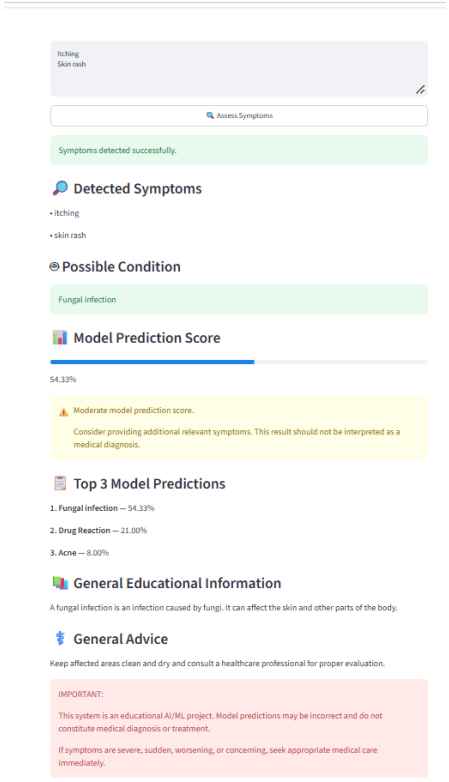
The application can be started from the VS Code terminal using the following command:

python -m streamlit run app.py

After running the command, Streamlit provides a local web address through which the application can be opened in a browser.

The application successfully connects the trained machine learning model with a simple user interface, making the project easier to demonstrate and understand.

Prediction result



ADVANTAGES

The AI Healthcare Symptom Assessment System provides several advantages as a machine learning based academic project. The system combines data preprocessing, machine learning, and a simple web application into one complete workflow.

15.1 Simple and Easy to Use

The application provides a simple interface through which users can enter their symptoms. Users do not need to understand machine learning or programming to interact with the system.

15.2 Quick Assessment

The trained model can generate a result within a short time after the symptoms are entered. Since the model is already trained and saved, the application does not need to repeat the training process for every prediction.

15.3 Multiple Symptoms

The system allows multiple symptoms to be considered together. This is useful because a combination of symptoms can provide more information to the machine learning model than considering only one symptom.

15.4 Machine Learning Based Approach

Instead of using only manually written rules, the system learns patterns from the available dataset. Different classification algorithms were also compared before selecting the final model.

15.5 Model Comparison

The project compares Decision Tree, Random Forest, Extra Trees, and K-Nearest Neighbors. This provides a better understanding of how different machine learning algorithms perform on the same dataset.

15.6 Reusable Trained Model

The final Random Forest model is saved using Joblib. The saved model can be loaded by the application whenever required, avoiding the need to train the model again every time the application is opened.

15.7 User-Friendly Output

The application displays the possible condition, prediction score, top three predictions, and basic educational information in a simple format. This makes the output easier to understand.

15.8 Practical Learning

The project provides practical experience in several areas of computer science, including data cleaning, feature engineering, classification algorithms, model evaluation, model saving, and web application development.

15.9 Educational Purpose

The system can be used as an academic demonstration of how machine learning can be applied to a healthcare-related dataset. It provides a complete example of converting a machine learning model into a working application.

Overall, the project demonstrates how a relatively simple machine learning workflow can be developed into an interactive application while keeping the interface easy to understand.

LIMITATIONS

Although the AI Healthcare Symptom Assessment System successfully demonstrates the use of machine learning for symptom-based assessment, the project has some limitations. These limitations are mainly related to the dataset, model evaluation, and the intended use of the application.

16.1 Dataset Size and Quality

The dataset used in this project contains a limited number of cleaned records compared with the amount of data that would normally be required for a real-world healthcare application. The quality and structure of the dataset directly affect the performance of the machine learning model.

16.2 Repeated and Similar Records

The original dataset contains repeated and very similar symptom patterns. After cleaning, the number of records was reduced significantly. This structure can make the classification problem easier for the machine learning models and can contribute to very high evaluation scores.

16.3 Limited Number of Symptoms

The system can only work with symptoms that are present in the dataset. If a user enters a symptom that was not included in the training data, the system cannot use that symptom effectively for prediction.

16.4 No Patient-Specific Information

The current system mainly considers symptoms. It does not consider other important factors such as age, gender, medical history, medications, laboratory test results, physical examination results, or other patient-specific information.

16.5 No Clinical Validation

The model has not been clinically validated by doctors or healthcare institutions. Therefore, the results obtained from the application cannot be considered medically reliable or suitable for making healthcare decisions.

16.6 Evaluation Results

Random Forest, Extra Trees, and K-Nearest Neighbors achieved 100% scores on the evaluated dataset. However, these results should not be interpreted as 100% accuracy in real-world healthcare situations. The performance is specific to the dataset and evaluation method used in this project.

16.7 Prediction Score

The prediction score displayed by the application is a model output based on the available data. It should not be interpreted as the actual probability that a user has a particular disease.

16.8 No Medical Diagnosis

The system is designed only as an educational symptom assessment application. It cannot replace professional medical consultation, physical examination, laboratory testing, or diagnosis by a qualified healthcare professional.

16.9 Internet and Application Environment

The current application is designed to run locally using Streamlit. It is not deployed as a production healthcare service and does not include features such as user accounts, cloud infrastructure, medical records, or integration with hospital systems.

These limitations show that although the project is useful for understanding the practical application

FUTURE SCOPE

The current project provides a basic machine learning based symptom assessment system. There are several ways in which the system can be improved in the future to make it more accurate, useful, and practical.

17.1 Larger and Better Dataset

The system can be trained using a larger and more diverse dataset containing a greater number of symptom combinations. Using high-quality and clinically validated data could help improve the reliability of the model.

17.2 More Machine Learning Algorithms

Additional machine learning and deep learning algorithms can be tested in future work. Algorithms such as Support Vector Machines, Gradient Boosting, XGBoost, neural networks, and other classification techniques could be compared with the current models.

17.3 Additional Patient Information

Future versions of the system could consider additional information such as age, gender, medical history, duration of symptoms, medications, and other relevant factors. This could provide the model with more information when generating an assessment.

17.4 Natural Language Input

The current application requires users to enter or select symptoms in a predefined format. A future version could allow users to describe their symptoms using normal sentences.

For example, instead of selecting individual symptoms, a user could enter a sentence such as "I have been experiencing fever, headache, and body pain since yesterday." Natural Language Processing techniques could then be used to identify the relevant symptoms.

17.5 Improved User Interface

The Streamlit interface can be further improved with better navigation, symptom search, categories, icons, visual indicators, and responsive design. This could make the application easier to use.

17.6 Explainable Predictions

Future versions could include explainable AI techniques to show why the model selected a particular condition. For example, the system could identify which symptoms contributed most strongly to the prediction.

This would make the model output easier to understand and would provide greater transparency.

17.7 Multilingual Support

The application could be extended to support multiple languages. This would allow users who are not comfortable with English to interact with the system more easily.

17.8 Cloud Deployment

The application could be deployed on a cloud platform so that it can be accessed through the internet. Proper security, privacy, and responsible data handling would be required before deploying any healthcare-related application.

17.9 Integration with Healthcare Systems

With appropriate validation, security, and professional oversight, future research could explore integration with electronic health records, laboratory results, or other healthcare information systems.

17.10 Professional and Clinical Validation

The most important future improvement would be testing the system using reliable clinical data and evaluating it with the involvement of qualified healthcare professionals. Extensive validation would be necessary before considering the system for any real-world healthcare application.

Overall, the future scope of this project is broad. The current system provides a foundation for exploring more advanced machine learning, natural language processing, explainable AI, and healthcare application development.

CONCLUSION

The AI Healthcare Symptom Assessment System was developed as a minor project to demonstrate how machine learning can be applied to a healthcare-related problem. The project focuses on assessing a combination of user-provided symptoms and identifying a possible condition based on patterns learned from the available dataset.

The project involved several stages, starting with dataset exploration and data cleaning. The symptoms were converted into numerical features using multi-hot encoding, while the disease names were converted into numerical labels using Label Encoding. These prepared features were then used to train and evaluate different machine learning classification algorithms.

Four algorithms were considered in the project: Decision Tree, Random Forest, Extra Trees, and K-Nearest Neighbors. Their performance was compared using accuracy, precision, recall, and F1-score. Random Forest, Extra Trees, and K-Nearest Neighbors achieved 100% scores on the evaluated dataset, while Decision Tree achieved comparatively lower results.

Based on the evaluation and its ensemble-based approach, Random Forest was selected as the final model. The final model was trained using the cleaned dataset and saved using Joblib. The saved model was then integrated into a Streamlit application.

The Streamlit application provides a simple interface for entering symptoms and viewing the model's output. It displays a possible condition, prediction score, top three predictions, and basic educational information. A medical disclaimer is also provided to make it clear that the application is not intended to provide a medical diagnosis.

One of the important outcomes of this project was understanding that high machine learning evaluation scores do not necessarily mean that a model will perform equally well in real-world situations. The dataset used in this project contains repeated and similar symptom patterns, and the model has not been clinically validated. Therefore, the results should be interpreted only within the context of this academic project.

Overall, the project provided practical experience in data preprocessing, feature engineering, machine learning model training, model evaluation, model saving, and web application development. It also demonstrated how concepts learned in computer science can be combined to build a complete machine learning application.

The developed system can serve as a foundation for future improvements involving larger datasets, natural language processing, explainable AI, additional patient information, and professional validation. With appropriate research, testing, and clinical oversight, these areas could be explored in future versions of the project.

REFERENCES

- Géron, A. (2022). Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow. 3rd Edition. O'Reilly Media.
    - Raschka, S., Liu, Y., & Mirjalili, V. (2022). Machine Learning with PyTorch and Scikit-Learn. Packt Publishing.
    - Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825–2830.
    - McKinney, W. (2022). Python for Data Analysis: Data Wrangling with pandas, NumPy, and Jupyter. 3rd Edition. O'Reilly Media.
    - Hunter, J. D. (2007). Matplotlib: A 2D Graphics Environment. Computing in Science & Engineering, 9(3), 90–95.
    - Streamlit Documentation. Streamlit – A faster way to build and share data applications.
    - Python Software Foundation. Python Documentation.
- Scikit-learn Documentation. Machine Learning algorithms, preprocessing, model evaluation, and classification documentation.
- Joblib Documentation. Python object persistence and model saving documentation.

- Pandas Documentation. Data analysis and data manipulation documentation.
- NumPy Documentation. Numerical computing and array processing documentation.
- Project Dataset. Disease and symptom dataset used for academic machine learning experimentation and model development.